Data Structures BS (CS)

_Fall_2025

Lab_10 Manual

Learning Objectives:

1. Understand the concept and structure of **Binary Search Trees (BSTs)**

2. Implement basic **BST operations** using **Linked Nodes**

3. Analyze **time complexities** of BST operations

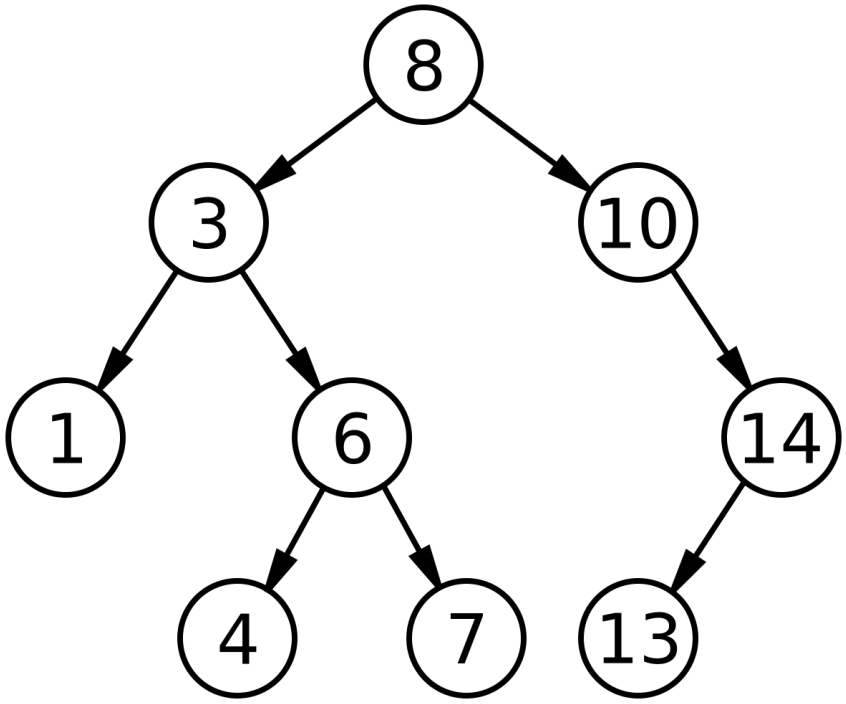
4. Apply traversal techniques (Inorder, Preorder, Postorder)

Binary Search Trees

A **Binary Search Tree (BST)** is a special kind of **binary tree** in which each node contains a unique and it maintains a specific order property:

- For every node:
    - The **left child** contains values **less than** the parent node.
    - The **right child** contains values **greater than** the parent node.

This property makes searching, insertion, and deletion efficient. Take a look at the visualization below:



Node Structure

A BST can be implemented using a linked structure where each node contains:

- `data` → stores the key or value
- `left` → pointer/reference to the left child
- `right` → pointer/reference to the right child

```
struct Node {

    int data;
```

```
    Node* left;

    Node* right;



    Node(int val) {

        data = val;

        left = right = nullptr;

    }

};
```

BST Operations

1. Insertion

Insertion follows these rules:
- If the tree is empty, create a new node as the root.
- If the key is less than the current node, go to the left subtree.
- If the key is greater, go to the right subtree.
- Continue recursively until you find the correct position.

```
Node* insert(Node* root, int key) {

    if (root == nullptr)

        return new Node(key);



    if (key < root->data)
```

```
        root->left = insert(root->left, key);

    else if (key > root->data)

        root->right = insert(root->right, key);



    return root;

}
```

2. Search

```
bool search(Node* root, int key) {

    if (root == nullptr)

        return false;

    if (root->data == key)

        return true;

    if (key < root->data)

        return search(root->left, key);

    else

        return search(root->right, key);

}
```

3. Traversals

BST supports three common depth-first traversals:

- Inorder (Left, Root, Right) → Yields elements in sorted order
- Preorder (Root, Left, Right) → Used for tree copying
- Postorder (Left, Right, Root) → Used for tree deletion

```
void inorder(Node* root) {

    if (root == nullptr) return;

    inorder(root->left);

    cout << root->data << " ";

    inorder(root->right);

}



void preorder(Node* root) {

    if (root == nullptr) return;

    cout << root->data << " ";

    preorder(root->left);

    preorder(root->right);

}



void postorder(Node* root) {
```

```
    if (root == nullptr) return;

    postorder(root->left);

    postorder(root->right);

    cout << root->data << " ";

}
```

4. Deletion

```
Node* findMin(Node* root) {

    while (root->left != nullptr)

        root = root->left;

    return root;

}



Node* deleteNode(Node* root, int key) {

    if (root == nullptr) return root;



    if (key < root->data)

        root->left = deleteNode(root->left, key);
```

```
    else if (key > root->data)

        root->right = deleteNode(root->right, key);

    else {

        // Node found

        if (root->left == nullptr) {

            Node* temp = root->right;

            delete root;

            return temp;

        }

        else if (root->right == nullptr) {

            Node* temp = root->left;

            delete root;

            return temp;

        }



        Node* temp = findMin(root->right);

        root->data = temp->data;

        root->right = deleteNode(root->right, temp->data);

    }
```

```
    return root;

}
```

Time Complexities

Operation	Best Case	Average Case	Worst Case	Description
Insert	O(log n)	O(log n)	O(n)	Insert element in correct position
Search	O(log n)	O(log n)	O(n)	Find an element by value
Delete	O(log n)	O(log n)	O(n)	Remove an element
Traversal	O(n)	O(n)	O(n)	Visit all nodes

Learning Outcome

After completing this lab, students should be able to:
- Construct and manipulate Binary Search Trees
- Implement recursive operations

- Perform and interpret tree traversals
- Analyze BST efficiency and recognize the importance of balanced trees